Abstraction

- Abstract Classes
- Interfaces

Interfaces

```
interface Car {
    void drive();
}
```

↳ Interfaces defines what an object can do without telling how it does that.

```
Class Car {
    void drive() {
        // do something
    }
}
```

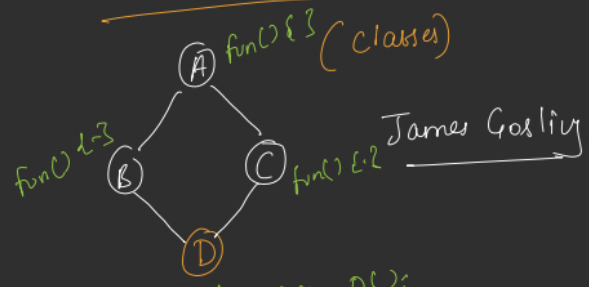
Blueprint of an Object

```
interface Car {
    void drive();
}
```

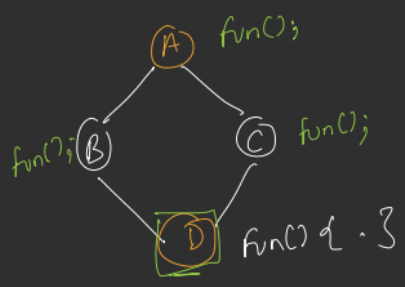
Blueprint of Behaviour

⇒ Contract

Diamond Problem




```
D d = new D();
d.fun();
```



[A, B, C are Interfaces]

D → class

Java 8 / 9 (Interface)

- default Methods
- static Methods
- private Methods

→ static & final variable

Abstract

- Concrete Method
- ——
- ——
- abstract

Interface

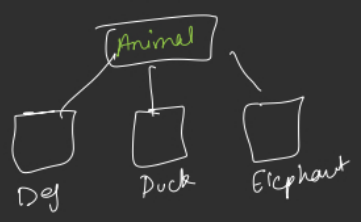
↳ Contract / Roles / functionalities

Eg: Runnable, Walkable, Payable (A)

(Can-do) Relationship

Abstract Class

↳ Families of similar class.



(IS-A) Relationship

	Interfaces	Abstract Classes
1	↳ Fields (X)	↳ Fields (✓)
	interface A { int x = 4; }	abstract class A { int x; }
2	Constructor (X)	Constructor (✓)
3	Multiple Inheritance (✓)	(X)
4	Methods → Public (✓), Private (✓)	Private, Public, Protected, dec → (✓)

Functional Interfaces

→ Functional programming

↓

Lambda Expressions

```
Interface A {
    void fun();
}

Class B implements A {
    void fun() {
    }
}
```

Comparable

Predicate ✓

Marker Interfaces

```
interface A {
}
```

→ Marker Interfaces

```
Interface Cloneable {
}
```

```
Interface Serializable {
}
```

```
Interface RandomAccess {
}
```

Object ⟷ clone();

Student implements Cloneable.

↳ clone() { }

```
interface Animal {
    void run();
}
```

javac Animal.java

↳ Animal.class